



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Implementación y Análisis de Algoritmos en Árboles (C++ y Java)
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero - B
Alumnos participantes:	León Tusa Jair Alexander
Asignatura:	Estructura de Datos
Docente:	Ing José Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Implementar y analizar algoritmos fundamentales para el manejo de árboles n-arios y binarios mediante programación orientada a objetos (C++ y Java), afianzando los conceptos teóricos de estructuras de datos no lineales.

Específicos:

- Desarrollar algoritmos de recorrido, inserción y transformación lógica en árboles.
- Aplicar la recursividad como técnica principal para la resolución de problemas topológicos.
- Comparar el manejo de estructuras en memoria (punteros en C++ frente a referencias en Java)

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 8

No presenciales: 0

2.4 Instrucciones

1. Lleve a cabo las actividades indicadas.
2. Suba a plataforma un informe con el resultado obtenido

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- **Inteligencia artificial, TAC**
- **PC de escritorio o portátil con JDK instalado e IDE de su elección**

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☐ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☒ Gamificación
- ☒ Inteligencia Artificial
- Otros (Especifique): _____

2.6 Actividades por desarrollar

Resolver los 5 ejercicios planteados en el entorno de desarrollo, completando la lógica faltante para conteo de nodos, inserción, cálculo de altura, recorridos y transformaciones, garantizando que la salida de consola coincida con los resultados esperados.

2.7 Resultados obtenidos



Los algoritmos fueron ejecutados satisfactoriamente en C++ y Java. A continuación, se detalla la lógica implementada para cada operación matemática y topológica:

1. Conteo de Nodos (Árbol N-ario): Se aplicó recursividad. La función valida el caso nulo (retorna 0). Si el nodo existe, inicializa un sumador en 1 y emplea un bucle para iterar la lista de hijos. Realiza una llamada recursiva por cada hijo, acumulando los subtotaes para obtener el conteo final.

EJECUCION

```
PS C:\Users\leonj\Downloads\APE3_ARBOLES> cd "c:\Users\leonj\Downloads\APE3_ARBOLES"
o app } ; if ($?) { .\app }
--- Prueba Ejercicio 1 ---
Nodos esperados: 6
Nodos calculados: 6
PS C:\Users\leonj\Downloads\APE3_ARBOLES\APE_Arboles\cpp>
```

2. Inserción en BST: El algoritmo se basa en el principio de partición binaria. Si el valor es menor al nodo actual, la recursión desciende por la rama izquierda; si es mayor o igual, desciende por la derecha. Al encontrar una referencia nula (hoja disponible), instancia el nuevo nodo sin romper el balance del árbol.

EJECUCION

```
PS C:\Users\leonj\Downloads\APE3_ARBOLES\APE_Arboles\cpp> cd "c:\Users\leonj\Downloads\APE3_ARBOLES\APE_Arboles\cpp"
?) { g++ *.cpp -o app } ; if ($?) { .\app }
--- Prueba Ejercicio 2 ---
Raiz (Esperado 10): 10
Hijo Izquierdo de Raiz (Esperado 5): 5
Hijo Derecho de Raiz (Esperado 15): 15
PS C:\Users\leonj\Downloads\APE3_ARBOLES\APE_Arboles\cpp>
```

3. Cálculo de Altura: La función recursiva navega hasta las ramas finales. Al retornar, mide de manera independiente la altura del subárbol izquierdo y derecho, extrae el máximo entre ambos y le suma 1 para representar la altura total acumulada por cada nivel.

EJECUCION

```
PS C:\Users\leonj\Downloads\APE3_ARBOLES\APE_Arboles\cpp> cd "c:\Users\leonj\Downloads\APE3_ARBOLES\APE_Arboles\cpp"
?) { g++ *.cpp -o app } ; if ($?) { .\app }
--- Prueba Ejercicio 3 ---
Altura esperada: 3
Altura calculada: 3
PS C:\Users\leonj\Downloads\APE3_ARBOLES\APE_Arboles\cpp>
```

4. Recorrido In-Order: Siguiendo el flujo simétrico (Izquierda -> Raíz -> Derecha), la función atraviesa el subárbol izquierdo primero. Al regresar al nodo intermedio, captura su valor en memoria, y finalmente explora el subárbol derecho. Esta técnica logró extraer los datos en orden ascendente en el BST.

EJECUCION

```
PS C:\Users\leonj\Downloads\APE3_ARBOLES> cd "c:\Users\leonj\Downloads\APE3_ARBOLES"
) { .\app }
--- Prueba Ejercicio 4 ---
Tu resultado: [1, 2, 3, 4, 5, 6, 7]
PS C:\Users\leonj\Downloads\APE3_ARBOLES\APE_Arboles\cpp>
```



5. Transformación (Invertir Árbol): El algoritmo espejo recorre la estructura intercambiando referencias directamente. Almacena temporalmente el hijo izquierdo, asigna el resultado de invertir la derecha al lado izquierdo, y viceversa, logrando una simetría invertida en todo el árbol.

EJECUCION

```
PS C:\Users\leonj\Downloads\APE3_ARBOLES\APE_Arboles\cpp> cd "c:\Users
o app } ; if ($?) { .\app }
--- Prueba Ejercicio 5 ---
Antes de invertir:
Hijo Izq: 2 | Hijo Der: 3

Despues de invertir:
Hijo Izq: 3 | Hijo Der: 2
PS C:\Users\leonj\Downloads\APE3_ARBOLES\APE_Arboles\cpp>
```

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☐ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☒ Flexibilidad
- ☐ La resolución de conflictos
- ☒ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

La abstracción de algoritmos de árboles requiere una conceptualización clara de la recursividad. Se verificó que, independientemente del paradigma de gestión de memoria utilizado por C++ (punteros manuales) o Java (recolector de basura), las estrategias de ruteo y lógicas de transformación mantienen la misma eficiencia temporal, operando de manera predecible.

2.10 Recomendaciones

Se recomienda escalar la práctica hacia la implementación de árboles auto-balanceables (como árboles AVL o Red-Black). Puesto que la inserción de datos ordenados en un BST simple puede causar degeneración, aplicar metodologías de balanceo garantizará tiempos de acceso $O(\log n)$ en todos los casos.

2.11 Referencias bibliográficas

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [2] M. A. Weiss, Data Structures and Algorithm Analysis in C++, 4th ed. Pearson, 2013.

2.12 Anexos

<https://github.com/JairLeonUTa/APE-3-.git>